

Synthetix Alpha

An autonomous, risk-gated trading agent on Alpaca

Alan Smith (King's College London) · Ciprian Mitoceanu (University of Bucharest) · Dhairya Sindhvani (Imperial College London)

Alpaca AI Trading Agents Hackathon, September 2026 · github.com/alan-d-smith/synthetix-alpha

AI logic

The system is built on one rule: LLM agents research, a deterministic runtime trades. Every session runs the same code on the same schedule with no model call in the order path, and the model earns its place in the two jobs where a model is actually better than a rule.

Research loop. Agents search arXiv's quantitative-finance categories on a paced API, score papers for relevance and download them into an append-only library. One agent per paper writes candidate strategies as declarative JSON specs rather than code: option legs by delta, moneyness or width; a DTE window; entry gates over 22 trailing-only features (IV surface, realised vol, VIX family, earnings and FOMC distance, technicals); exits; sizing. The brief carries what is already known, forbids tuning on the backtest, and treats gains under 0.54 Sharpe as noise. A backtester replays real end-of-day option chains day by day, filling at mid plus half the spread, charging \$0.65 per contract and leg, refusing contracts with no same-day volume, carrying positions through splits and settling at intrinsic. Candidates are scored on $\frac{1}{2}\bar{S} + \frac{1}{2}S_{\min} + 2 \cdot \text{worst year} + 3 \cdot \text{max drawdown} + (\text{positive years} - 1)$, void under 40 trades, then re-run on an unseen vendor (DoltHub surfaces, 2019–2026), an unseen underlying, and a one-at-a-time parameter perturbation sweep. Results go to an append-only progress log that keeps corrections even when the score falls. Promotion to live is a human step. The loop produced the deployed options rule over five generations and 140+ candidates, and it later re-measured the intraday sleeve on minute bars and reported that its backtested edge was an artefact of the opening print.

Critic. In the operator dashboard a Featherless-hosted model, prompted as a risk officer, reviews each screened candidate with FintHub news and FRED macro context and returns a JSON decision: approve or reject, a confidence, and a size multiplier. It is advisory by construction. Approvals below 70 confidence are dropped, every approval is re-run through the deterministic risk gate, and an operator confirms in two steps before a paper order exists.

Risk gates

Every order, from the scheduler or the dashboard, passes the same code before it reaches the broker.

- **Account** (`governance.yaml`): NAV must be positive; halt at 20% total or 5% daily drawdown; leverage $1\times$; 10% of NAV per name; 12 open positions; defined-risk structures only; 3% of equity at risk per position, measured on the payoff grid.
- **Screen:** IV/RV between 1.25 and 2; price above \$5; \$20M average daily volume; option-chain spread under 10% of mid, measured from OptionMetrics; no earnings within 30 days.
- **Order:** the credit must be at least 5% of max loss, and the credit re-priced at bid and ask must clear 60% of the mid credit, so a spread that only looks rich at mid never trades; each contract needs 25 trades that day; whole contracts only.

- **Intraday:** a corporate-action guard refuses gaps beyond 25%; the ranking refuses a stale session; exits are sized to the actual fill, so an unfilled entry never leaves a naked exit; flattening covers only what is held and skips resting orders, in two passes at 15:50 and 15:56.
- **Windows:** entries 09:31–10:30 ET, flattening 15:30–15:58, weekends refused; a live run outside the window is refused outright.
- **Idempotency:** the client order id is a hash of the legs and the date, checked in a local ledger and against the broker's open, closed and all orders, so a retry or a restart cannot double-fill. Status is never reported as filled unless Alpaca said so.
- **Paper only:** the runner asserts paper mode and strips the live-trading switch from every child environment; the dashboard refuses live keys with a 403.

Alpaca infrastructure

Alpaca is used through two transports, on purpose.

Trading goes through the Alpaca CLI. Every account read and every order is a subprocess call to the `alpaca` binary: `account get`, `position list`, `order list`, `order get`, `order cancel`, `option contracts` and `order submit`. Spreads are submitted as multi-leg limit orders (`-order-class mleg` with a JSON legs payload, per-leg `position_intent`, up to four legs, day time in force); the equity basket and crypto sleeve use market orders. The wrapper parses the CLI's JSON, raises on its error envelope and never lets the environment opt into live trading. The dashboard's Docker image builds the CLI from source so Render runs the same transport as the VPS.

Market data goes through the `alpaca-py` SDK. Backtests, the intraday ranking and the options screen are bulk data jobs: split-adjusted daily panels for 198 names, one-minute bars for 1,233 sessions pulled at four sessions a second, full option-chain snapshots with implied vol and greeks on the indicative feed, and hourly crypto bars. The SDK's historical clients batch and page these requests; the CLI is a per-call tool and is not the right instrument for them. The SDK's trading client is used only to read option-contract metadata. This split keeps the hackathon's CLI requirement where it matters, on the order path, and uses the SDK where it is the better engineering choice.

Runtime. A scheduler on a Hostinger VPS fires at 09:31, 09:45, 15:50 and 15:56 ET, each in a clean subprocess with its own credentials, marking state before every fire so a crash never retries; a cron watchdog restarts it within five minutes. Market-on-close orders are placed as insurance and the real exit is a market order at 15:50, because the paper engine does not honour MOC. Resting sells are cancelled before liquidation to avoid wash-trade rejections. The competition runs on a dedicated \$100k paper account opened for the event; its ID is listed with the entry.